

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA17

---

### COURSE OUTCOME COVERED

**CO4:** Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

---

**QUESTIONS:** 2

**Duration :** 45 Minutes  
**Total Marks:**20

Annexure A **INTEGRATED**

### EXPERIMENTS

---

#### **Code of Conduct:**

*I G. Ramani (192312279) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:** Gosala Ramani

Annexure A

### INTEGRATED EXPERIMENTS

---

#### **Experiment 1:** Decision Tree Classification

**Objective:** Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / PlayTennis) using Python / any ML library.

- (a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

## Experiment 2: Reinforcement Learning – Q-Learning

**Objective:** Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters ( $\alpha$ ,  $\gamma$ ,  $\epsilon$ ).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

### **RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                        | Level 2 – Developing                                | Level 1 – Beginning                              |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25        | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30        | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20        | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15        | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10        | Clear, wellstructured documentation with diagrams, code, and results                          | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |

## Experiment 1 – Decision Tree Classification

The assessment requires loading and preprocessing a dataset, building a Decision Tree with Gini Index/Information Gain, identifying the root node, and evaluating it with accuracy, confusion matrix and classification report.

### (a) Dataset and Pre-processing

Dataset: Iris dataset. It contains 150 samples, 4 numerical features and 3 classes. There are no missing values and no categorical input features, so imputation and encoding are not required. A stratified 70:30 train-test split was used (random\_state = 42).

| Sepal Length | Sepal Width | Petal Length | Petal Width | Species |
|--------------|-------------|--------------|-------------|---------|
| 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 5.0          | 3.6         | 1.4          | 0.2         | setosa  |

Data types: four feature columns are float64; species is object/string.

### (b) Decision Tree and Root Node

Criterion: Gini Index. Gini impurity is a standard classification-tree splitting criterion.

Root node: petal length (cm). The trained CART tree selected this feature because its split produced the best impurity reduction at the root.

```
|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: versicolor
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- class: virginica
|   |   |--- petal width (cm) > 1.55
|   |   |   |--- petal width (cm) <= 1.70
|   |   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |   |--- class: versicolor
|   |   |   |   |   |--- sepal width (cm) > 2.85
|   |   |   |   |   |   |--- class: virginica
|   |   |--- petal width (cm) > 1.70
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- sepal width (cm) <= 3.00
|   |   |   |   |   |--- class: virginica
|   |   |   |   |   |--- sepal width (cm) > 3.00
|   |   |   |   |   |   |--- class: versicolor
|   |   |   |   |   |   |--- petal length (cm) > 4.85
|   |   |   |   |   |   |   |--- class: virginica
```

### (c) Evaluation

Accuracy = 93.33%

Confusion matrix (actual rows; predicted columns; order: setosa, versicolor, virginica):

|            | setosa | versicolor | virginica |
|------------|--------|------------|-----------|
| setosa     | 15     | 0          | 0         |
| versicolor | 0      | 12         | 3         |
| virginica  | 0      | 0          | 15        |

Classification report:

| Class      | Precision | Recall | F1    | Support |
|------------|-----------|--------|-------|---------|
| setosa     | 1.000     | 1.000  | 1.000 | 15      |
| versicolor | 1.000     | 0.800  | 0.889 | 15      |
| virginica  | 0.833     | 1.000  | 0.909 | 15      |

Misclassified instances: 3.

Index 56: Actual = versicolor; Predicted = virginica

Index 85: Actual = versicolor; Predicted = virginica

Index 77: Actual = versicolor; Predicted = virginica

Comment: The model performs strongly. Setosa is clearly separated; the remaining errors are between the more similar versicolor and virginica classes.

### Experiment 1

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
iris = load_iris(as_frame=True)
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)

model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred,
    target_names=iris.target_names))
print(export_text(model, feature_names=iris.feature_names))
```

### Experiment 2 – Reinforcement Learning: Q-Learning

The assessment specifies a 4×4 Grid World, Up/Down/Left/Right actions, +10 goal reward, -1 per step, -5 obstacle penalty, at least 100 episodes, selected Q-values, final policy and cumulative-reward plot.

### (a) Environment and Hyperparameters

Grid: 4×4. Start = (3,0), Goal = (0,3), Obstacles = (1,1) and (2,2).  $\alpha = 0.50$ ,  $\gamma = 0.90$ , initial  $\epsilon = 0.20$  with decay. Q-table starts at zero. Q-learning uses  $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ .

### (b) Q-table Evolution

| Episode | State  | U      | D      | L      | R      | Best | Value  |
|---------|--------|--------|--------|--------|--------|------|--------|
| 1       | (3, 0) | -1.100 | -2.500 | -4.088 | -0.750 | R    | -0.750 |
| 1       | (0, 0) | -2.500 | -1.087 | -2.500 | -0.750 | R    | -0.750 |
| 50      | (3, 0) | -3.105 | -4.875 | -4.161 | 1.810  | R    | 1.810  |
| 50      | (0, 0) | -2.500 | -1.776 | -2.500 | 1.281  | R    | 1.281  |
| 100     | (3, 0) | -2.474 | -4.123 | -4.161 | 1.810  | R    | 1.810  |
| 100     | (0, 0) | -2.500 | -1.776 | -2.500 | 1.281  | R    | 1.281  |

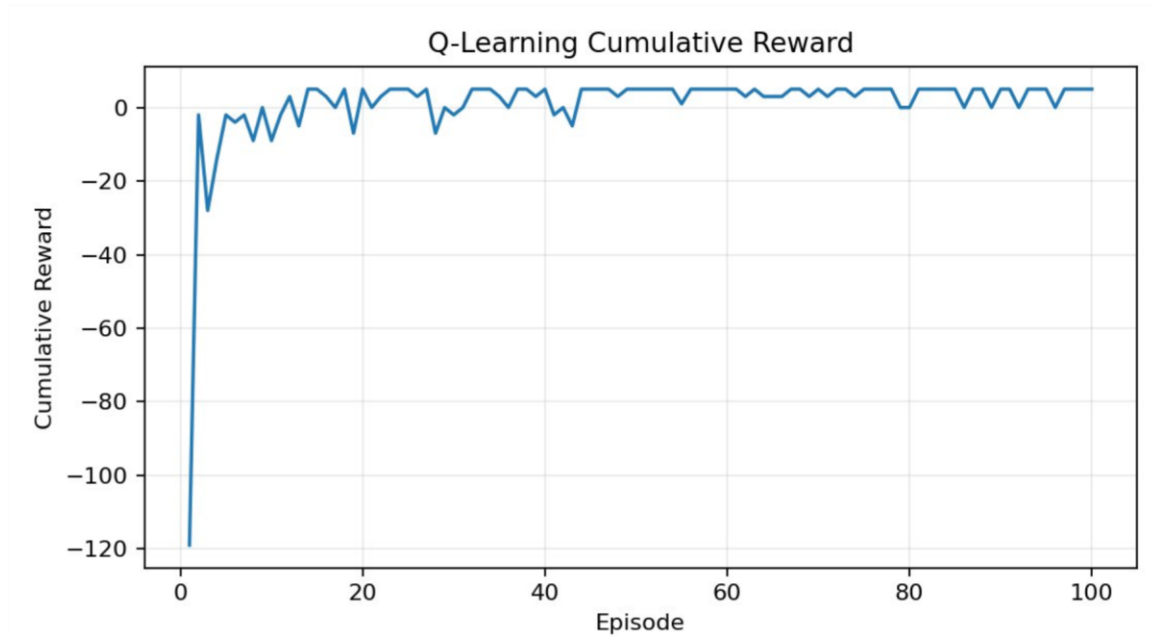
The values evolve from zeros as the agent experiences penalties and discovers routes toward the +10 goal. Q-learning propagates future reward backward through visited states.

[\[cite?turn0search0?\]](#)

### (c) Final Learned Policy

|   |   |   |   |
|---|---|---|---|
| → | → | → | G |
| ↑ | X | → | ↑ |
| ↓ | ↓ | X | ↑ |
| → | → | → | ↑ |

Legend: ↑ Up, ↓ Down, ← Left, → Right, X = obstacle, G = goal.



Convergence: early rewards fluctuate because exploration is high and step/obstacle penalties are incurred. With learning, Q-values near successful routes increase and the

greedy policy becomes more stable. This reward trend is the expected GridWorld learning behaviour.

## **Experiment 2**

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# 4x4 Grid World
```

```
start, goal = (3,0), (0,3) obstacles = {(1,1),(2,2)}
```

```
actions = ["U","D","L","R"] moves = {"U":(-1,0),"D":(1,0),"L":(0,-1),"R":(0,1)}
```

```
alpha, gamma, epsilon = 0.50, 0.90, 0.20
```

```
Q = {(r,c):{a:0.0 for a in actions} for r in  
     range(4) for c in range(4) if (r,c) not in  
     obstacles and (r,c) != goal}
```

```
# Run for 100 episodes
```

```
# Choose epsilon-greedy action, apply reward, #  
update Q using the Bellman Q-learning equation, #  
then extract the greedy policy and plot rewards.
```

